

Bubble Sort



Why is it named bubble sort?

- The data 'bubbles' up to the top of the dataset.



What is bubble sort?

A simple sorting algorithm that sorts elements in an unordered array:

1. Pass through the unsorted array, **comparing adjacent elements**. If the adjacent elements are in the wrong order, swap them until the end is reached.

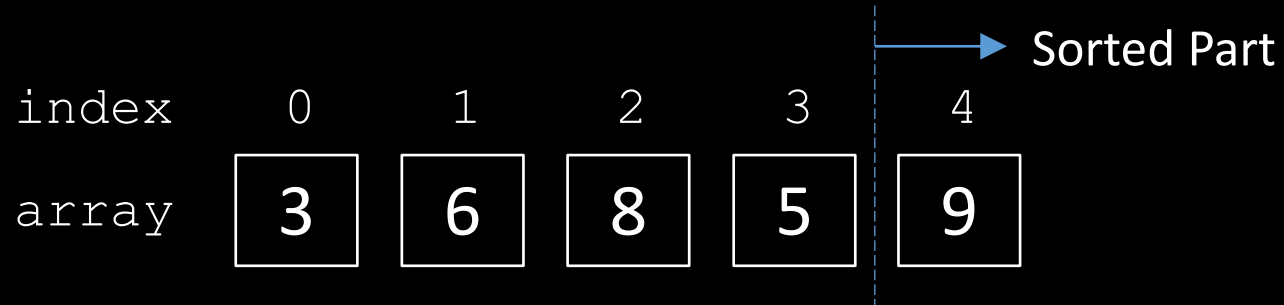
index	0	1	2	3	4
array	8	3	6	9	5

What is bubble sort?

A simple sorting algorithm that sorts elements in an unordered array:

1. Pass through the unsorted array, **comparing adjacent elements**. If the adjacent elements are in the wrong order, swap them until the end is reached.

First Pass

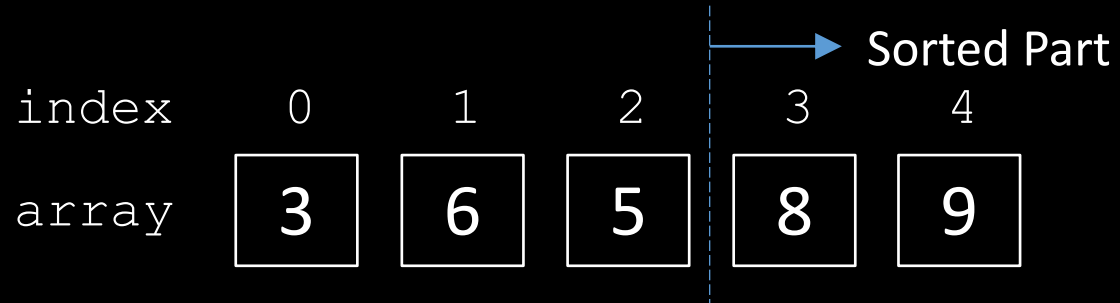


What is bubble sort?

A simple sorting algorithm that sorts elements in an unordered array:

1. Pass through the unsorted array, **comparing adjacent elements**. If the adjacent elements are in the wrong order, swap them until the end is reached.
2. Keep repeating passes on the unsorted part of the array, until the entire array is sorted.

Second Pass

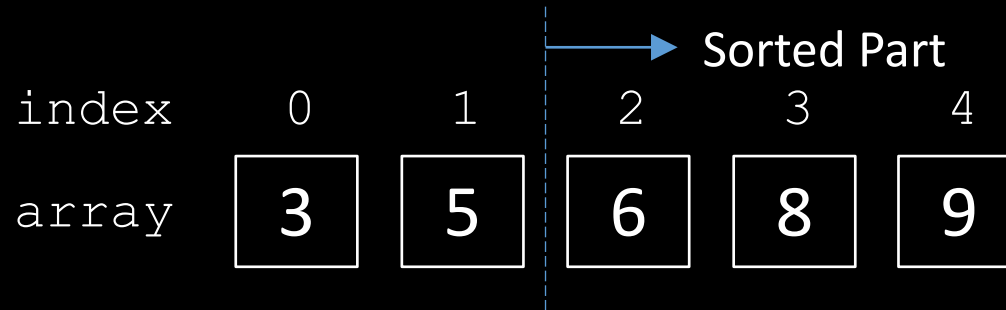


What is bubble sort?

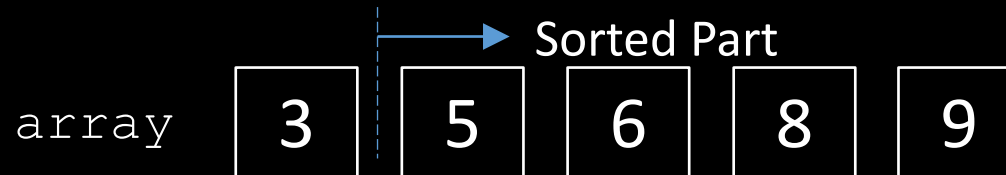
A simple sorting algorithm that sorts elements in an unordered array:

1. Pass through the unsorted array, **comparing adjacent elements**. If the adjacent elements are in the wrong order, swap them until the end is reached.
2. Keep repeating passes on the unsorted part of the array, until the entire array is sorted.

Third Pass



Fourth Pass



Sort Complete

Bubble Sort Inner Loop

Inner Loop:

- Compare adjacent element pairs, from the start.
i=0 24, 5, 36, 18, 12 (Check 24 and 5)
- If the elements are in the wrong order, swap them.
5, 24, 36, 18, 12 (Swap 24 and 5)
- Iterate this process until the end is reached.
i=1 5, 24, 36, 18, 12 (Check 24 and 36, no need to swap)
- i=2 5, 24, 18, 36, 12 (Check 18 and 36, swap)
- i=3 5, 24, 18, 12, 36 (Check 12 and 36, swap)
- The largest element will now be behind. Inner loop complete.

Bubble Sort Outer Loop

Outer Loop:

- Calls the inner loop to place the largest element behind.
5, 24, 18, 12, 36 (After 1 pass of the inner loop)
- Repeats the inner loop to place the 2nd largest element behind.
5, 18, 12, 24, 36 (After 2 passes of the inner loop)
- ...
- 5, 12, 18, 24, 36 (After 4 passes of the inner loop)
- A size n array will require $n - 1$ passes to sort the whole array

Optimised Bubble Sort

Outer Loop:

- Calls the inner loop to place the largest element behind.
3, 4, 5, 24, 18, 12, 36 (After 1 pass of the inner loop)
- Repeats the inner loop to place the 2nd largest element behind.
3, 4, 5, 18, 12, 24, 36 (After 2 passes of the inner loop)
- Repeat the inner loop to place the 3rd largest element behind.
3, 4, 5, 12, 18, 24, 36 (After 3 passes of the inner loop)
- Repeats the inner loop, but no swaps were required. Stop.
- Result: Only 3 passes required to sort the array,
And a 4th pass to ascertain that it is sorted. (Pass 5 and 6 can be skipped)

Is bubble sort in-place?

in-place: sorted items occupy the same storage as the original items

Bubble sort is in-place:

- Elements doing mutual swap **within the list**
- List is mutated
- No additional memory space needed to store another sorted list while sorting

Is bubble sort stable?

- What does it mean to be stable?
- Consider a case where there are repeated elements within an unsorted list.
- [3, 1, 4, 5, 3, 6, 2]
- Stable:
- [1, 2, 3, 3, 4, 5, 6] #relative position remains unchanged
- Unstable:
- [1, 2, 3, 3, 4, 5, 6] #swap occurs for elements with same value
- Stability: Depends on its implementation, not the type of sort used (did the algorithm swap when 2 elements are equal?)

Order of growth bubble sort

- Average case: $O(n^2)$
 - Generally uses nearly all $n - 1$ passes (Pass 1 to pass $n-1$)
 - Each pass does $(n - \text{pass number})$ comparisons
 - Total no. of comparisons: Sum of AP: $(n-1) + (n-2) + (n-3) + \dots + 1$
- Worse case: $O(n^2)$
 - List is arranged in a reversed manner
 - Swapping occurs at every step
- Best case: $O(n)$
 - List nearly/already sorted, optimized bubble sort: $(n-1) + (n-2)$

Advantages/Disadvantages

Advantages:

- Memory efficient
- Can implement stable sort easily
- For nearly sorted data sets, optimised bubble sort can perform at $O(n)$ time

Disadvantage:

- $O(n^2)$ has a big penalty and is not suitable for large data sets
- Time complexity $O(n^2)$ not as good as other sorting algorithms